

GII Global Infotech India Pvt. Ltd.

DevOps Engineer Interview — Question Bank & Answer Guide

Candidate: Sanket Ajay Chopade | Interview: Today 2:30 PM | Location: Nagpur (WFO)

■ HR & Introduction

Q1.

Tell me about yourself.

MODEL ANSWER

I'm Sanket Chopade, a fresher DevOps Engineer with 9 months of hands-on internship experience at Hisan Labs, Pune. During that time I worked on provisioning AWS infrastructure using Terraform, building CI/CD pipelines with Jenkins, and deploying microservices on Kubernetes EKS. My flagship project — ApnaKart — is a 10-microservice e-commerce platform I built end-to-end: from Terraform modules to Jenkins pipelines to Datadog APM monitoring. I reduced deployment time by 77% and maintained 99.9% uptime. I'm now looking to bring that same problem-solving mindset to a production environment at GII, where I can grow fast and contribute on real projects from day one.

■ *Interviewer Tip: Lead with impact numbers, not just tool names. End with why this company.*

Q2.

Why GII Global Infotech? Why Nagpur?

MODEL ANSWER

GII's emphasis on live production projects and a 'no hand-holding' culture aligns exactly with how I learn best — I don't want a sandbox, I want real stakes. Nagpur is my home city, and I want to contribute to the local tech ecosystem while growing fast. I also noticed GII works across cloud infrastructure and automation, which maps directly to my ApnaKart and Flight Reservation project experience.

■ *Interviewer Tip: Research GII's LinkedIn/website before the interview to add one specific detail about their work.*

Q3.

What is your expected CTC and notice period?

MODEL ANSWER

I'm open to a fair offer based on the role's responsibilities and growth potential. As a fresher with real production experience, I'd expect compensation aligned with the Nagpur DevOps fresher market — typically 3–5 LPA. I have no notice period and can join immediately.

■ *Interviewer Tip: Don't undersell yourself. 9 months of internship + production-grade project justifies 3–4 LPA minimum.*

Q4.

What is your biggest weakness?

MODEL ANSWER

I don't yet have an AWS certification, which I know is a benchmark many teams use. I'm actively preparing for the AWS Solutions Architect Associate exam. That said, I compensate with hands-on depth — I've actually provisioned EKS clusters, configured VPCs, and set up IAM with least-privilege policies in real projects, not just labs.

■ *Interviewer Tip: Always pair a weakness with what you're doing about it. Avoid clichés like 'I work too hard.'*

■ Linux / Ubuntu Administration

Q5.

How do you check which process is consuming the most CPU/memory on a Linux server?

MODEL ANSWER

I use 'top' or 'htop' for an interactive real-time view — press 'P' to sort by CPU, 'M' for memory. For a one-shot snapshot I use: `ps aux --sort=-%cpu | head -10`. For memory: `ps aux --sort=-%mem | head -10`. In production I also check with 'iotop' for disk I/O bottlenecks alongside CPU. During my internship I wrote a Bash script to alert via Slack webhook when CPU > 80% on our EC2 instances.

■ *Interviewer Tip: Show you go beyond the basic command — mention real usage context.*

Q6.

What is the difference between a hard link and a soft (symbolic) link?

MODEL ANSWER

A hard link points directly to the inode of a file — it's an additional directory entry for the same data block. Deleting the original file doesn't break a hard link because the inode still has references. A soft link (symlink) is a pointer to the file path, not the inode. If the original file is deleted, the symlink breaks. Hard links can't span filesystems or link to directories (usually). Practical use: I use symlinks for config files in multi-environment setups — e.g., symlinking `/etc/app.conf` to the environment-specific config file.

■ *Interviewer Tip: Practical use-case earns extra marks in DevOps interviews.*

Q7.

How do you troubleshoot a server that is unreachable via SSH?

MODEL ANSWER

I follow a layered approach: (1) Check if the instance is running — AWS EC2 console or 'ping' test. (2) Verify Security Group rules — port 22 open for my IP. (3) Check NACL rules at subnet level. (4) Verify the key pair — using the correct .pem file. (5) Check sshd status via EC2 Instance Connect or Systems Manager Session Manager if SSH is blocked. (6) Review `/var/log/auth.log` or `/var/log/secure` for failed auth attempts. This exact checklist saved us once when someone accidentally modified the NACL during a Terraform apply.

■ *Interviewer Tip: Interviewers love real-world stories. Prepare one per topic.*

Q8.

Explain the Linux boot process briefly.

MODEL ANSWER

BIOS/UEFI → POST → Bootloader (GRUB2) → Kernel loads and decompresses into memory → Kernel mounts the root filesystem → init/systemd starts as PID 1 → systemd brings up services in parallel based on unit files → Login prompt or graphical interface appears. Key files: `/boot/grub/grub.cfg`, `/etc/systemd/system/`, `/etc/fstab` for mount points.

■ *Interviewer Tip: For DevOps, focus on the systemd layer — interviewers often ask follow-up about service management.*

Q9.

What is chmod 755 and chmod 644? When do you use them?

MODEL ANSWER

chmod 755: Owner can read/write/execute; Group and Others can read/execute. Used for scripts and directories that need to be accessible but not modified by others — e.g., deployment scripts. chmod 644: Owner can read/write; Group and Others can only read. Used for config files, HTML files, or any file that should be readable but not executable — e.g., `nginx.conf`. In my Terraform automation scripts I set executable permissions using 'chmod +x' before running them in Jenkins pipelines.

■ *Interviewer Tip: Always connect permissions to a real DevOps context — config files, scripts, SSH keys.*

■ Git & Version Control

Q10.

What is the difference between git merge and git rebase?

MODEL ANSWER

git merge combines two branches and creates a new 'merge commit', preserving the full history of both branches. It's non-destructive but creates a messier log. git rebase moves or replays commits from one branch on top of another, resulting in a linear history. It rewrites commit hashes, so it should only be used on local/feature branches, never on shared branches like main. In my Jenkins CI/CD pipeline, I used 'git merge' for production branches and 'git rebase' for cleaning up feature branches before raising PRs.

■ *Interviewer Tip: The 'never rebase shared branches' rule is a common follow-up trap question.*

Q11.

How do you resolve a merge conflict in Git?

MODEL ANSWER

When a conflict occurs: (1) Run git status to see conflicting files. (2) Open the file — Git marks conflicts with <<<<<<, =====, >>>>>> markers. (3) Edit the file to keep the correct code and remove the markers. (4) Run git add to mark it resolved. (5) Run git commit to complete the merge. For complex conflicts I use 'git mergetool' with vimdiff or VS Code as the diff tool. I also use 'git log --merge' to understand which commits caused the conflict.

■ *Interviewer Tip: Walk through each step confidently — this is a very common practical question.*

Q12.

What is git stash and when would you use it?

MODEL ANSWER

git stash temporarily saves your uncommitted changes without committing them, so you can switch branches or pull updates without losing work. Commands: 'git stash' saves changes. 'git stash list' shows all stashes. 'git stash pop' restores the latest stash and removes it. 'git stash apply' restores without removing. Real use: During my internship, I was mid-way through a config change when a production hotfix was needed. I stashed my changes, checked out the hotfix branch, fixed it, then popped my stash to continue.

■ *Interviewer Tip: The real-world story makes this answer memorable.*

■ Docker & Containerization

Q13.

What is the difference between a Docker image and a Docker container?

MODEL ANSWER

A Docker image is a read-only, layered blueprint — it includes the OS base, dependencies, and application code. Think of it as a class in OOP. A Docker container is a running instance of that image — it adds a writable layer on top. Think of it as an object instantiated from that class. You can run multiple containers from the same image. Containers are ephemeral by default — data is lost unless you use volumes.

■ *Interviewer Tip: The class/object analogy is clean and easy for interviewers to follow.*

Q14.

What is a multi-stage Docker build and why do you use it?

MODEL ANSWER

Multi-stage builds use multiple FROM statements in one Dockerfile. The first stage (builder) compiles or builds the application. The second stage (runtime) copies only the built artifact — not the build tools — into a minimal base image like Alpine or distroless. Result: drastically smaller final images. In my Flight Reservation project, I cut image sizes by 60% using this approach. Example: Stage 1 uses 'maven:3.9-jdk17' to build the Spring Boot JAR. Stage 2 uses 'eclipse-temurin:17-jre-alpine' and just copies the JAR — no Maven, no JDK bloat in production.

■ *Interviewer Tip: Your 60% image reduction metric is a great proof point — use it.*

Q15.

What is the difference between COPY and ADD in a Dockerfile?

MODEL ANSWER

COPY simply copies files from the host to the container — straightforward and preferred for most cases. ADD does everything COPY does, but also: auto-extracts .tar.gz archives and can fetch files from remote URLs. Best practice: Always use COPY unless you specifically need ADD's extraction or URL features. ADD with a URL is considered a security risk because it can pull arbitrary remote content at build time.

■ *Interviewer Tip: Knowing the security implication of ADD makes you stand out from candidates who just list the difference.*

Q16.

How do Docker volumes differ from bind mounts?

MODEL ANSWER

Volumes are managed by Docker — stored in /var/lib/docker/volumes/. They're portable, can be backed up easily, and work across containers. Best for databases and persistent app data. Bind mounts map a specific host path directly into the container. Useful in development to live-edit code without rebuilding the image, but tightly coupled to the host filesystem structure. In production I use named volumes for database persistence and bind mounts only in dev environments.

■ *Interviewer Tip: Production vs dev distinction shows maturity — interviewers at real-project companies value this.*

■ Kubernetes (K8s)

Q17.

Explain the Kubernetes architecture — control plane vs worker node.

MODEL ANSWER

Control Plane components: API Server (all communication goes through it), etcd (distributed key-value store for cluster state), Scheduler (assigns pods to nodes based on resources), Controller Manager (runs controllers like Deployment, ReplicaSet, Node controllers). Worker Node components: kubelet (agent that ensures containers run as per PodSpec), kube-proxy (handles network routing and service IPs), Container Runtime (Docker/containerd). In my EKS setup, AWS manages the control plane. I managed worker node groups using managed node groups in Terraform — defining instance types, min/max capacity, and IAM roles.

■ *Interviewer Tip: Connecting architecture knowledge to your actual EKS experience is powerful.*

Q18.

What is the difference between a Deployment, StatefulSet, and DaemonSet?

MODEL ANSWER

Deployment: Manages stateless applications. Pods are interchangeable. Supports rolling updates and rollbacks. I used Deployments for all 10 Spring Boot microservices in ApnaKart. StatefulSet: For stateful applications like databases. Each pod has a stable, unique network identity and persistent storage. Pods are created and deleted in order. DaemonSet: Ensures one pod runs on every node (or selected nodes). Used for cluster-wide tools. I deployed the Datadog agent as a DaemonSet in ApnaKart so every node's metrics were captured automatically.

■ *Interviewer Tip: The Datadog DaemonSet example directly matches your resume — use it confidently.*

Q19.

What is a Kubernetes Ingress and how is it different from a LoadBalancer Service?

MODEL ANSWER

A LoadBalancer Service provisions a cloud load balancer (like AWS ALB/NLB) per service — expensive and unscalable if you have 10+ services. Ingress is a Layer 7 (HTTP/HTTPS) routing resource that uses a single Ingress Controller (like NGINX or AWS ALB Ingress Controller) to route traffic to multiple services based on host or path rules. In ApnaKart I used a single Ingress with path-based routing: /api/cart → cart-service, /api/orders → order-service, etc. This saved cost and reduced complexity.

■ *Interviewer Tip: Cost and complexity justification shows production thinking, not just textbook knowledge.*

Q20.

What is HPA (Horizontal Pod Autoscaler) and how does it work?

MODEL ANSWER

HPA automatically scales the number of pod replicas based on observed metrics — CPU utilization, memory, or custom metrics. It queries the Metrics Server every 15 seconds and compares current usage to the target threshold. Formula: $\text{desired replicas} = \text{ceil}(\text{current replicas} \times (\text{current metric} / \text{target metric}))$. I configured HPA in both ApnaKart and Flight Reservation — setting CPU target at 70% to absorb 5x peak load automatically without manual intervention. Important: HPA requires Metrics Server to be deployed in the cluster.

■ *Interviewer Tip: Knowing the 15-second sync interval and the formula differentiates you from candidates who just say 'it auto-scales.'*

Q21.

How do you rollback a Kubernetes deployment?

MODEL ANSWER

Quick rollback: `kubectl rollout undo deployment/` — rolls back to the previous revision. To rollback to a specific version: `kubectl rollout undo deployment/ --to-revision=`. Check rollout history: `kubectl rollout history deployment/`. In my Jenkins pipeline I added a post-deployment health check — if the readiness probe failed within 2 minutes, the pipeline automatically triggered a rollback using `'kubectl rollout undo'`. This gave us 98% deployment success rate.

■ *Interviewer Tip: Automated rollback in CI/CD pipeline ties Kubernetes to your internship achievement directly.*

■ CI/CD — Jenkins & GitHub Actions

Q22.

Walk me through the CI/CD pipeline you built at Hisan Labs.

MODEL ANSWER

Our pipeline for ApnaKart had these stages: (1) Code Push → GitHub webhook triggers Jenkins. (2) Maven Build — compile and run unit tests. (3) SonarQube static analysis — code quality gate; pipeline fails if critical issues found. (4) Docker image build with multi-stage Dockerfile. (5) Trivy image scan — pipeline stops if critical CVEs found. (6) Push image to DockerHub (dev/test) or ECR (uat/prod). (7) Kustomize overlays applied per environment. (8) `kubectl apply` to EKS namespace. (9) Health check — verify readiness probes; auto-rollback on failure. This reduced deployment time from 45 minutes to 10 minutes — a 77% reduction.

■ *Interviewer Tip: This is your strongest answer. Practice delivering it in under 90 seconds.*

Q23.

What is the difference between Jenkins Pipeline (Scripted) and Declarative Pipeline?

MODEL ANSWER

Scripted Pipeline uses Groovy syntax directly — full power and flexibility but complex to read and maintain. Declarative Pipeline uses a structured, opinionated syntax with a `'pipeline {}'` block. It's easier to read, supports built-in directives like `'agent'`, `'stages'`, `'post'`, and has better error reporting. I used Declarative Pipelines in my internship because they're team-friendly and readable by non-Groovy developers. I only dropped to Scripted for advanced loops when doing dynamic environment matrix builds.

■ *Interviewer Tip: Knowing when to use each is the mature answer — not just listing differences.*

Q24.

What are GitHub Actions and how do they compare to Jenkins?

MODEL ANSWER

GitHub Actions is a cloud-native CI/CD platform built into GitHub. Pipelines are defined as YAML workflows in `.github/workflows/`. No server to maintain — GitHub hosts runners. Jenkins is self-hosted (unless using Jenkins Cloud), highly customizable via plugins, and language-agnostic. GitHub Actions: faster setup, free for public repos, tight GitHub integration, marketplace actions. Jenkins: better for complex enterprise pipelines, on-prem requirements, and fine-grained control. I used GitHub Actions for a smaller project with simpler needs and Jenkins for ApnaKart due to its complexity and the need for custom Kubernetes agents.

■ *Interviewer Tip: Demonstrating you've used both and made a reasoned choice shows experience, not just exposure.*

■ Cloud — AWS

Q25.

Explain VPC and its key components.

MODEL ANSWER

A VPC (Virtual Private Cloud) is a logically isolated network in AWS. Key components: Subnets (public for internet-facing resources, private for internal), Route Tables (control traffic routing), Internet Gateway (allows public subnets to reach internet), NAT Gateway (allows private subnets to initiate outbound internet traffic without being exposed), Security Groups (stateful instance-level firewall), NACLs (stateless subnet-level firewall). In ApnaKart I created a custom VPC with 2 public subnets (ALB, NAT) and 4 private subnets (EKS nodes, RDS) across 2 AZs for high availability — all provisioned via Terraform.

■ *Interviewer Tip: Real Terraform architecture beats textbook definitions every time.*

Q26.

What is IAM and what is the principle of least privilege?

MODEL ANSWER

IAM (Identity and Access Management) controls who can do what in AWS. Key components: Users, Groups, Roles, and Policies. Principle of Least Privilege: grant only the minimum permissions needed to perform a task — nothing more. In practice: EKS node group gets only the permissions to pull from ECR and write to CloudWatch Logs. Jenkins gets only the permissions to push images to ECR and update EKS deployments — not full admin. I created custom IAM policies in Terraform with specific Action/Resource combinations rather than using AWS managed AdministratorAccess.

■ *Interviewer Tip: The Jenkins IAM restriction example shows you apply least privilege in real pipelines.*

Q27.

What is the difference between Security Groups and NACLs?

MODEL ANSWER

Security Groups: Stateful (return traffic is automatically allowed). Applied at the instance/ENI level. Supports only allow rules — no deny. Evaluated before traffic reaches the instance. NACLs: Stateless (you must explicitly allow both inbound and outbound). Applied at the subnet level. Supports both allow AND deny rules. Rules evaluated in order by rule number — first match wins. Practical rule: Security Groups for instance-level control, NACLs for broad subnet-level blocks (e.g., blocking a malicious IP range).

■ *Interviewer Tip: Stateful vs stateless is the core concept — explain it first, then add the details.*

Q28.

What is the difference between S3 Standard, S3-IA, and S3 Glacier?

MODEL ANSWER

S3 Standard: Frequently accessed data. High availability, low latency. Most expensive storage. S3 Standard-IA (Infrequent Access): For data accessed less than once a month. Lower storage cost but retrieval fee applies. Minimum 30-day storage charge. S3 Glacier: For archival data (accessed rarely, like once a year). Very cheap storage, retrieval takes minutes to hours. In my project, I stored Terraform state files in S3 Standard with versioning enabled, and archived old application logs to S3-IA after 30 days using lifecycle policies.

■ *Interviewer Tip: Lifecycle policy mention shows you think about cost optimization — GII's JD values this.*

■ Terraform (IaC)

Q29.

What is the difference between terraform plan and terraform apply?

MODEL ANSWER

terraform plan: Dry-run that compares your configuration with the current state and shows what changes will be made — create, modify, or destroy. It does NOT make any changes. terraform apply: Actually executes the plan and makes changes to the infrastructure. Best practice: Always run 'plan' first and review carefully before 'apply'. In my CI/CD pipeline I ran 'terraform plan' in the PR stage for human review, and 'terraform apply -auto-approve' only on merge to the main branch in a controlled environment.

■ *Interviewer Tip: Separating plan (review) from apply (execute) is a production-grade practice — say it.*

Q30.

What is Terraform state and why is it important?

MODEL ANSWER

Terraform state is a JSON file (terraform.tfstate) that maps your configuration to real infrastructure resources. It's the source of truth for what Terraform has created. Without state, Terraform can't know what exists — it would try to recreate everything. Best practices: Never store state locally in a team environment. Use remote backend (S3 + DynamoDB for locking). Enable versioning on the S3 bucket for state recovery. In ApnaKart I configured S3 backend with DynamoDB state locking to prevent concurrent applies from corrupting the state file.

■ *Interviewer Tip: DynamoDB locking is a commonly missed detail — mentioning it shows production experience.*

Q31.

What are Terraform modules and why do you use them?

MODEL ANSWER

A module is a reusable, self-contained package of Terraform configuration. Instead of repeating VPC/EKS/RDS code across environments, you write it once as a module and call it with different input variables. In ApnaKart I built modules for: VPC, EKS cluster, RDS, and IAM roles. Calling the VPC module with different CIDR blocks and tags for dev/test/uat/prod environments reduced my Terraform codebase by ~60% and ensured consistency across environments.

■ *Interviewer Tip: The 60% reduction metric and multi-environment example is strong. Use it.*

■ Monitoring & Observability

Q32.

What is the difference between Prometheus and Grafana?

MODEL ANSWER

Prometheus is a time-series metrics collection and storage system. It scrapes metrics from exporters (node_exporter, kube-state-metrics) and stores them. It also handles alerting via AlertManager. Grafana is a visualization and dashboarding platform. It queries Prometheus (and other data sources) and displays metrics in charts, graphs, and heatmaps. Think of it as: Prometheus = database + scraper. Grafana = the dashboard on top. In my internship I set up Prometheus with kube-state-metrics to scrape pod/node metrics and built Grafana dashboards with CPU, memory, and request latency panels for each microservice.

■ *Interviewer Tip: The analogy (database + dashboard) makes complex concepts digestible for non-technical interviewers.*

Q33.

What is Datadog and how did you use it in ApnaKart?

MODEL ANSWER

Datadog is a SaaS observability platform covering infrastructure monitoring, APM (Application Performance Monitoring), log management, and synthetic testing in a single platform. In ApnaKart I deployed the Datadog agent as a DaemonSet on EKS so it automatically collected host-level metrics from every node. I also used Datadog APM with distributed tracing — tagging traces with service name, version, and environment. This allowed me to trace a single user request across all 10 microservices and pinpoint where latency was introduced. It helped us reduce MTTR by 30% by cutting root-cause identification from 20 minutes to under 5 minutes.

■ *Interviewer Tip: The distributed tracing story and the MTTR number are your strongest monitoring proof points.*

■ DevSecOps — Security Practices

Q34.

What is Trivy and what does it scan?

MODEL ANSWER

Trivy is an open-source vulnerability scanner by Aqua Security. It scans: Container images (OS packages + language-level dependencies), Filesystems and Git repositories, Kubernetes manifests and Helm charts (misconfiguration detection), Terraform IaC files. In my Jenkins pipeline: after building the Docker image, Trivy scans it. If any CRITICAL severity CVE is found, the pipeline fails and the image is not pushed. This ensured zero critical vulnerabilities reached production across all ApnaKart deployments.

■ *Interviewer Tip: Failing the pipeline on CRITICAL CVEs is the mature DevSecOps answer — it shows you shift security left.*

Q35.

What is SonarQube and what does it check?

MODEL ANSWER

SonarQube is a static code analysis platform. It checks for: Bugs (code likely to cause runtime errors), Vulnerabilities (security issues like SQL injection patterns, hardcoded secrets), Code Smells (maintainability issues, duplications, complex methods), Test Coverage (percentage of code covered by unit tests). In my pipeline, SonarQube runs after Maven build. I configured a Quality Gate — if coverage drops below 70% or any new blocker issues are found, the pipeline stops. This maintained consistent code quality across all 10 microservices in ApnaKart.

■ *Interviewer Tip: Quality Gate configuration detail shows you use SonarQube properly, not just as a badge.*

■ Scenario-Based & Problem-Solving Questions

Q36.

A production deployment just failed and the application is down. Walk me through your response.

MODEL ANSWER

Step 1 — Immediate rollback: `kubectl rollout undo deployment/` to restore the last known good state. Step 2 — Verify recovery: Watch pod status with `'kubectl get pods -w'`. Check readiness probes. Step 3 — Investigate: `kubectl logs --previous` for crash logs. `kubectl describe pod` for events. Step 4 — Check monitoring: Datadog/Grafana for error spike, latency increase, memory/CPU anomaly. Step 5 — Root cause: Was it a bad image? Config change? Resource limit? Infrastructure issue? Step 6 — Fix forward: Address root cause, test in staging, redeploy. Step 7 — Post-mortem: Document what failed, why, and what runbook was updated to prevent recurrence. The goal is to minimize blast radius first, then investigate — never the other way around.

■ *Interviewer Tip: The 7-step structure + post-mortem shows SRE thinking. Interviewers love this.*

Q37.

A Kubernetes pod is stuck in CrashLoopBackOff. How do you debug it?

MODEL ANSWER

CrashLoopBackOff means the container starts, crashes, and Kubernetes keeps restarting it. Debug steps: (1) `kubectl describe pod` — check Events section for OOMKilled, ImagePullBackOff, liveness probe failures. (2) `kubectl logs --previous` — logs from the previous (crashed) container instance. (3) Check resource limits — OOMKilled means the container exceeded its memory limit. (4) Check liveness probe config — probe might be too aggressive or misconfigured. (5) Check image — verify the image tag exists and was built correctly. (6) Check environment variables / secrets — missing config causes app startup failures. Most common cause in my experience: misconfigured env vars or a DB connection string pointing to the wrong host.

■ *Interviewer Tip: Listing OOMKilled and liveness probe as specific causes shows real debugging experience.*

Q38.

Your CI/CD pipeline is running but deployments are getting slower over time. What do you investigate?

MODEL ANSWER

I'd check each stage for bottlenecks: (1) Docker build — large image? Use multi-stage builds and `.dockerignore`. Use BuildKit cache. (2) Test stage — slow unit tests? Parallelize them in Jenkins. (3) Image push — slow network to registry? Switch to ECR in the same region. (4) `kubectl apply` — cluster under load? Check API server latency. (5) Jenkins agent — shared agent with too many concurrent jobs? Add more agents or use K8s-native Jenkins pods. I'd also check Jenkins build logs for queue wait times vs actual execution time — often the slowdown is queueing, not execution.

■ *Interviewer Tip: Distinguishing queue wait time from execution time is a subtle but impressive observation.*

■ Questions YOU Should Ask the Interviewer

1. What does the first 30/60/90 days look like for this role?
2. What cloud provider and tools does GII primarily use in production?
3. What does the on-call or incident response structure look like for DevOps engineers here?
4. What are the biggest infrastructure or automation challenges the team is currently facing?
5. How does the team handle knowledge sharing — documentation, runbooks, internal wikis?
6. Is there a defined growth path from fresher/junior DevOps to senior or SRE?

■ *Tip: Asking 2–3 smart questions signals genuine interest and seniority of thinking. Avoid asking about salary, leave policy, or WFH in the first interview.*